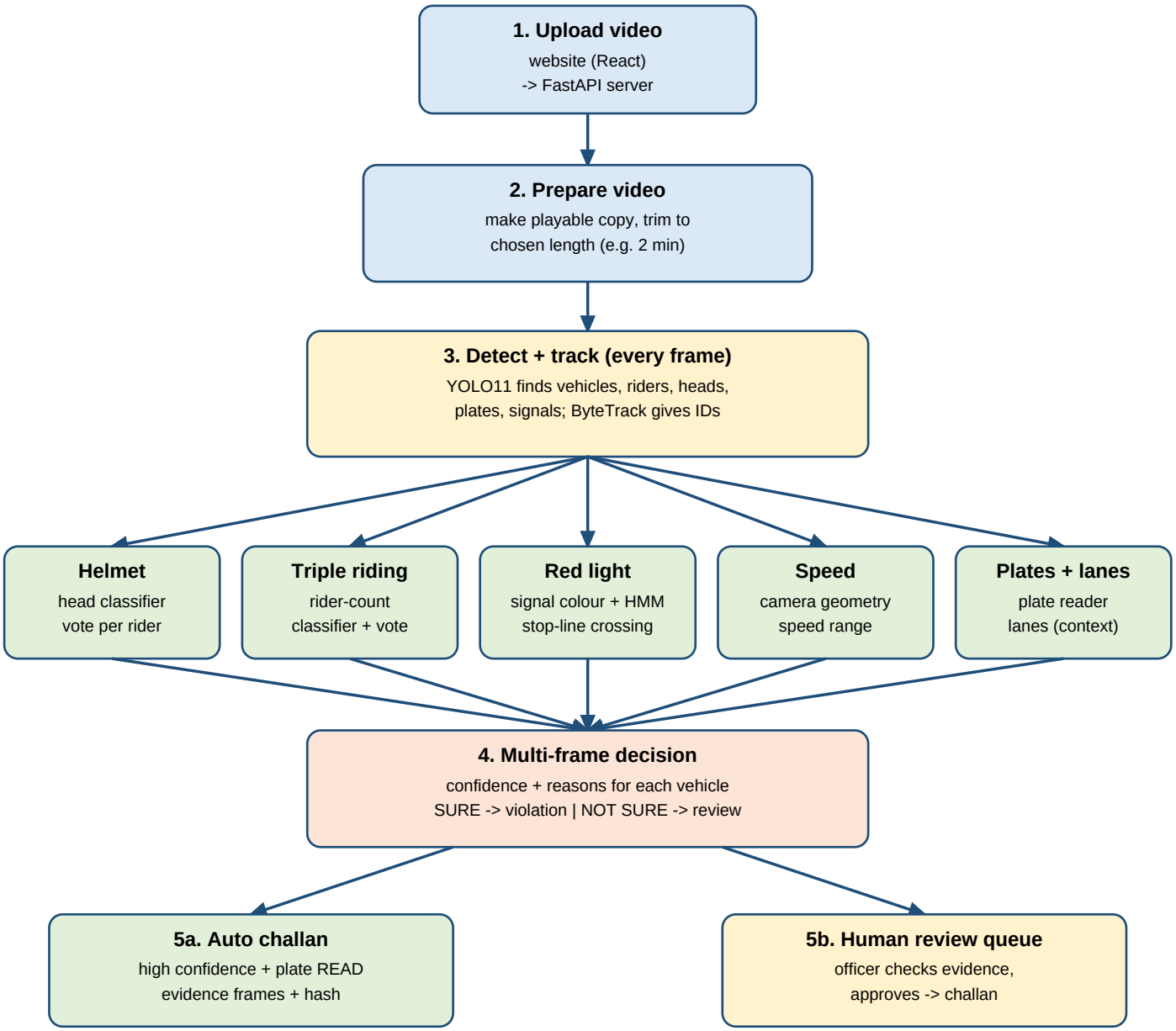
Traffic Violation Detection System — explained simply

This document explains how the system works: the main idea, how multi-frame decisions are made, how lanes are found, how each violation is checked, and how the code is organised. Read the diagram first, then the sections below.

Main flow of the system



Every check follows the same pattern: **detect** things in each frame, **track** them over time, **vote** across many frames, then give a **decision with a confidence and reasons**.

1. The main technique (used everywhere)

A single video frame is often misleading: a head can be blurred, a signal can look dark for one frame, a plate can be half hidden. So the system never decides from one frame. It uses four steps:

• **Detect** – a YOLO11 neural network looks at each frame and draws boxes around vehicles, riders, heads, number plates and traffic lights. A model trained on Indian roads (IDD dataset) is used for vehicles.

• **Track** – ByteTrack links the boxes from frame to frame so each vehicle keeps the same ID (vehicle #12 in frame 1 is still #12 in frame 50).

• **Classify** – small specialised models answer one question each: helmet or not? how many riders? is the lamp red, yellow or green? what characters are on the plate?

• **Decide** – the answers from all frames of the same vehicle are combined (multi-frame voting) into one decision: VIOLATION, NOT SURE or OK, with a confidence number and written reasons (for explainable AI).

Safety rule: the system is built to avoid wrong challans. If it is not sure, it says NOT SURE and a human reviews the case, instead of guessing.

2. Multi-framing (multi-frame voting)

Once a vehicle is tracked, the system has many views of it, for example 60 frames over 2 seconds. Each frame gives a probability (e.g. 'no helmet: 0.9'). These are combined:

• **Weighted vote** – clear, large views count more than small or blurry ones (helmet, triple riding).

• **Two thresholds** – above the high threshold = violation; below the low threshold = OK; in between = NOT SURE. The thresholds were chosen on test videos the models never saw, so that wrong calls are close to zero.

• **Smoothing over time** – for traffic lights a Hidden Markov Model is used: a lamp keeps its colour for seconds and changes in the order green → yellow → red, so one misread frame cannot flip the state.

• **Agreement check** – for number plates, a plate is only accepted (READ) if several sharp frames agree on the same text and it matches the Indian format (e.g. MH 12 AB 1234). Otherwise it is NOT SURE.

Example: a rider is seen in 40 frames; 34 frames say 'no helmet' with high confidence, 6 are blurry. The weighted vote is 0.93 > threshold → 'No helmet', confidence 0.93, reason 'vote over 40 frames, clearest head 62 px tall'.

3. Lane detection (LaneTrace)

Indian roads often have faded or missing lane paint, so lanes are found from **how the traffic drives**, not from paint:

• **Observe** – detect and track all vehicles; each frame is aligned to a reference frame so a shaking pole camera still gives clean paths (a camera pan or cut starts a new segment).

• **Road coordinates** – perspective makes far lanes look narrow. The system finds the horizon and converts every vehicle position to 'road coordinates', where every lane has the same width near and far.

• **Find lanes** – where vehicles drive most often forms peaks; the spacing of the peaks gives the lane width. If there is no lane discipline (common in India), lanes = used road width ÷ standard lane width (3.5 m), shown as '~N'.

• **Confidence** – depends on how many vehicles were seen and how clearly they form lanes. Needs at least 30 s of video (60 s recommended). Lanes are shown as context only and never create a challan.

4. How each violation is checked

Violation	How it works (simple)	Output
No helmet	Find motorcycles and the people on them, crop each rider's head, a YOLO11 classifier says helmet / no helmet in every frame, then a size-weighted vote per rider.	No helmet / NOT SURE / OK. Extra persons on a bike always go to review (could be a bystander).
Triple riding	For each two-wheeler, a classifier counts riders (and heads on the seat) in many frames and votes.	Triple riding only if sure; else NOT SURE.
Red-light jumping	Signal lamps are found and their colour read every frame (red/yellow/green + HMM smoothing). The stop line and controlled direction are learned from traffic (where vehicles queue at red). A vehicle crossing the line while red for > 1 s = violation. If the light never turns green in the video, crossings go to review.	VIOLATION / NOT SURE / EXEMPT (ambulance, police, fire) / OK
Emergency exemption	A CLIP image model scores each crossing vehicle as ambulance / police / fire.	Emergency vehicles are never challaned.
Over-speeding	Camera geometry (horizon, lens, camera height from typical vehicle sizes) converts pixel movement to metres; speed is repeated ~300 times with small variations to give a 90 % range.	Speed range + confidence; needs a calibration file.
Number plate	Plate detector (YOLO) + a plate reader fine-tuned on Indian plates (81.6 % exact on unseen plates). Characters voted across frames, checked against Indian format.	READ (used on challan) / NOT SURE (human types it)

From finding to challan: confident violations with evidence (frames before/at/after, box, reasons, file hash) become challans automatically; NOT SURE cases wait in the review queue for a human decision. Repeated uploads of the same footage are recognised so the same vehicle is not challaned twice.

5. Structure of the code

```
traffic violation system/
|-- app/                        the website (main product)
|   |-- start.sh                ./start.sh  -> http://127.0.0.1:8600
|   |-- server/                 FastAPI back-end (Python)
|   |   |-- main.py             web API: login, upload, findings, review, challans
|   |   |-- pipeline.py         runs the checks on a video, makes evidence + challans
|   |   |-- ml/detectors.py     one Detector class per check (helmet, triple,
|   |   |                       red light, speed, lanes) - calls the tools below
|   |   |-- ml/plate.py         number-plate reader wrapper
|   |-- web/                    React + TypeScript front-end (pages: Dashboard,
|   |                           Analysis, Review, Challans, Settings, VideoGuide)
|   |-- data/                   database (platform.db) + one folder per analysis
|-- helmet_training/robust/     helmet model training + run_video.py
|-- triple_training/            triple-riding model + run_video.py
|-- red_light/                  observe.py (detect/track/signals) + decide.py
|-- ambulance_training/         emergency-vehicle (CLIP) probe
|-- speed_estimation/           track.py -> calibrate.py -> speed.py
|-- plates/                     plate reader training + run_video.py
|-- lane_trace/                 lane detection from traffic
|-- runs/, models/              trained model weights
|-- videos_final/               test videos
```

Adding a new violation = write one new Detector class in `app/server/ml/detectors.py`; the website, evidence and review queue work for it automatically.

6. Honest limits

• Needs a fixed camera, daylight, 720p or better; far or blurry vehicles go to NOT SURE.

• Plates are read only when large and sharp enough (about 100 px wide); otherwise a human enters them.

• Red light needs the signal's lit face visible to the camera; clips without a visible signal report nothing.

• Speed needs one known road length (calibration) to be accurate.